UNIVERSIDADE FEDERAL DE OURO PRETO

***CAMPUS* MORRO DO CRUZEIRO – OURO PRETO**

BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

Alunos:

Bernardo Fernandes Costa

João Pedro Cavalcante

Maria Paula Miranda de Souza Montovani Gasparini

Mateus Lima de Araújo

Vítor de Sousa Costa Lopes

Desenvolvimento do jogo TERMO em Java com Interface Gráfica (Swing)

12/02/2026

Professor: Guilhermo Camara Chavez

SUMÁRIO

1. INTRODUÇÃO

    1.1. Objetivo Geral

    1.2. Objetivos Específicos

    1.3. Escopo do sistema desenvolvido

    1.4 Descrição do sistema

    1.5 Quais entidades o sistema manipula

    1.6 Como o usuário interage com o sistema via terminal

2. DESENVOLVIMENTO

    2.1. Especificação de Requisitos

    2.2. Diagrama de Classes

3. DECISOẼS DE PROJETO

4. TESTES E TRATAMENTO DE ERROS

5. CONCLUSÃO

INTRODUÇÃO

Este trabalho apresenta o desenvolvimento do jogo **TERMO**, inspirado no Wordle em português, utilizando a linguagem Java e a biblioteca Swing para a construção da interface gráfica. O projeto foi concebido no contexto da disciplina de Programação Orientada a Objetos (POO), com o objetivo de aplicar, de forma prática, conceitos fundamentais como encapsulamento, herança, polimorfismo, associação entre classes e manipulação de arquivos.

O sistema desenvolvido permite ao usuário interagir com uma interface gráfica intuitiva, na qual deve descobrir uma palavra secreta de cinco letras em até seis tentativas, recebendo feedback visual por meio de cores que indicam se as letras estão corretas, em posição incorreta ou ausentes na palavra.

.

1.1 Objetivo Geral

O objetivo principal deste trabalho é desenvolver o jogo TERMO em Java, utilizando a biblioteca Swing para interface gráfica, implementando corretamente os conceitos de Programação Orientada a Objetos e garantindo o funcionamento completo das regras do jogo, incluindo leitura de banco de palavras, controle de tentativas e armazenamento de estatísticas.

1.2 Objetivos Específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

- Implementar classes que representem as principais entidades do sistema, como Jogo, Partida, Palavra, Jogador e Tentativa, aplicando corretamente o encapsulamento.

- Utilizar herança e polimorfismo quando aplicável, organizando adequadamente a hierarquia de classes.

- Desenvolver a interface gráfica utilizando componentes da biblioteca Swing (como JFrame, JButton, JLabel, JTextField, JPanel), garantindo interatividade adequada com o usuário.

- Implementar a leitura do banco de palavras a partir de um arquivo texto informado como argumento na execução do programa.

- Desenvolver a lógica do jogo, incluindo:

    - Seleção aleatória de palavras;

    - Controle de tentativas (máximo de 6);

    - Validação de palavras digitadas;

    - Geração de feedback visual com cores (verde, amarelo e cinza).

- Implementar o controle de estatísticas, contabilizando número de partidas jogadas, vencidas e perdidas.

- Garantir organização modular do código, boas práticas de programação e comentários explicativos.

1.3 Escopo do sistema desenvolvido

O escopo do sistema abrange o desenvolvimento completo do jogo TERMO com interface gráfica, contemplando tanto a lógica do jogo quanto a interação visual com o usuário.

O software permite iniciar novas partidas, selecionar aleatoriamente palavras a partir de um banco de palavras carregado de arquivo texto, processar tentativas do jogador e fornecer feedback visual por meio de cores (verde, amarelo e cinza), indicando o estado de cada letra.

O sistema também realiza o controle de estatísticas do jogador, contabilizando partidas realizadas, vencidas e perdidas, além de gerenciar o estado atual da partida (tentativas restantes e status de vitória ou derrota).

A aplicação opera por meio de uma interface gráfica construída com a biblioteca Swing, permitindo interação por meio de botões, campos de texto e componentes personalizados (como os quadrados coloridos que representam cada letra digitada). O banco de palavras é carregado a partir de um arquivo externo informado na execução do programa, garantindo flexibilidade e separação entre dados e lógica.

1.4 Descrição do sistema

O sistema foi implementado com separação clara entre camada de controle, lógica de jogo e interface gráfica, conforme verificado nos arquivos Principal.java, ControladorJogo.java e JanelaPrincipal.java.

As principais funcionalidades do sistema incluem:

Início e Controle de Partidas

- O arquivo Principal.java é responsável por iniciar a aplicação e carregar o banco de palavras.

- O ControladorJogo gerencia o fluxo da partida, selecionando a palavra secreta, controlando o número de tentativas e verificando o estado da partida (EstadoPartida).
- O sistema garante que cada partida possua no máximo 6 tentativas.

Processamento de Tentativas

- A classe AvaliadorTentativa é responsável por comparar a palavra digitada com a palavra secreta.

- O resultado da análise é armazenado em ResultadoTentativa, que associa cada letra a um estado (EstadoLetra), indicando:
    - Letra correta na posição correta (verde);
    - Letra existente em posição incorreta (amarelo);
    - Letra inexistente na palavra (cinza).

Interface Gráfica

- A classe `JanelaPrincipal` implementa a interface gráfica utilizando componentes Swing.

- Os quadrados coloridos que representam as letras são implementados pela classe QuadradoLetra.

- A padronização visual das cores é centralizada na classe `TemaCores`.

- A interface permite:
    - Inserção de palavras pelo jogador;
    - Visualização gráfica das tentativas;
    - Exibição de tentativas restantes;
    - Botões para nova partida e saída do jogo.

Banco de Palavras

- A classe `BancoPalavras` é responsável por ler o arquivo texto contendo palavras válidas.
- O sistema seleciona aleatoriamente palavras ainda não utilizadas anteriormente.

Estatísticas

- A classe Estatisticas armazena informações sobre:
    - Total de partidas jogadas;
    - Total de vitórias;
    - Total de derrotas.
- Esses dados são atualizados automaticamente ao final de cada partida.

Estados e Utilidades

- A classe EstadoPartida controla o status atual do jogo (em andamento, vencida ou perdida).

- A classe UtilTexto auxilia no tratamento de texto, como normalização de entrada do usuário.

1.5 Quais entidades o sistema manipula

O sistema manipula as seguintes entidades, representadas por classes no código:

- **Jogo**: Responsável por controlar a lógica geral do jogo, incluindo seleção da palavra secreta, controle de tentativas e verificação do estado da partida
- **BancoPalavras**: Responsável por carregar e armazenar as palavras válidas a partir de um arquivo texto externo informado na execução do programa.

- **Tentativa**: Representa uma palavra digitada pelo jogador durante uma rodada.
- **AvaliadorTentativa**: Classe responsável por comparar a palavra digitada com a palavra secreta e gerar o resultado correspondente.
- **ResultadoTentativa**: Armazena o resultado da avaliação de cada letra, associando-a a um estado específico.
- **EstadoLetra**: Enumeração que representa o estado de cada letra (CORRETA, POSICAO_INCORRETA, AUSENTE).
- **EstadoPartida**: Enumeração que controla o status da partida (EM_ANDAMENTO, VENCIDA, PERDIDA).
- **JanelaPrincipal**: Classe responsável pela interface gráfica do sistema, utilizando componentes da biblioteca Swing.
- **Estatisticas**: Responsável por armazenar o número total de partidas jogadas, vitórias e derrotas.
- **QuadradoLetra**: Representa visualmente cada letra digitada na interface, sendo responsável por exibir as cores correspondentes.

1.6 Como o usuário interage com o sistema via terminal

A interação ocorre por meio de uma interface gráfica construída com a biblioteca Swing.

O usuário:

- Digita uma palavra de cinco letras em um campo de texto (JTextField);
- Confirma a tentativa por meio de botão ou tecla Enter;
- Visualiza o resultado da tentativa por meio de quadrados coloridos organizados em grade (GridLayout);
- Acompanha o número de tentativas restantes exibido na tela;
- Pode iniciar uma nova partida por meio do botão “Nova Partida”;
- Pode encerrar o jogo por meio do botão “Sair”, momento em que as estatísticas finais são exibidas.

Diferentemente do sistema anterior baseado em terminal, toda a interação ocorre de forma visual e dinâmica.

DESENVOLVIMENTO

2.1 Especificação de Requisitos

Com base no enunciado proposto para o desenvolvimento do jogo TERMO,o sistema atende aos seguintes requisitos funcionais e não funcionais:

Requisitos Funcionais Implementados:

- **RF01 - Iniciar Partida:** O sistema seleciona aleatoriamente uma palavra de cinco letras do banco de palavras carregado.
- **RF02 - Processar Tentativas:** O jogador pode inserir até seis palavras válidas por partida.

● **RF03 - Avaliação de Letras:** O sistema compara cada letra da tentativa com a palavra secreta e retorna o estado correspondente (verde, amarelo ou cinza).

● **RF04 - Controle de Estatísticas:** O sistema contabiliza partidas jogadas, vencidas e perdidas.

● **RF05 - Nova Partida:** O usuário pode reiniciar o jogo a qualquer momento.

● **RF06 - Encerramento do Jogo:** O sistema exibe as estatísticas finais ao sair.

● **RF07 - Leitura de Banco de Palavras:** O sistema carrega as palavras a partir de arquivo texto informado como argumento na linha de comando.

- **Requisitos Não Funcionais:**

  ● Linguagem de implementação: Java.

  ● Interface: Interface gráfica utilizando biblioteca Swing.

  ● Organização: Estrutura modular orientada a objetos.

  ● Armazenamento: Leitura de arquivo texto contendo palavras válidas.

  ● Usabilidade: Interface simples, intuitiva e funcional.

2.2 Diagrama de Classes

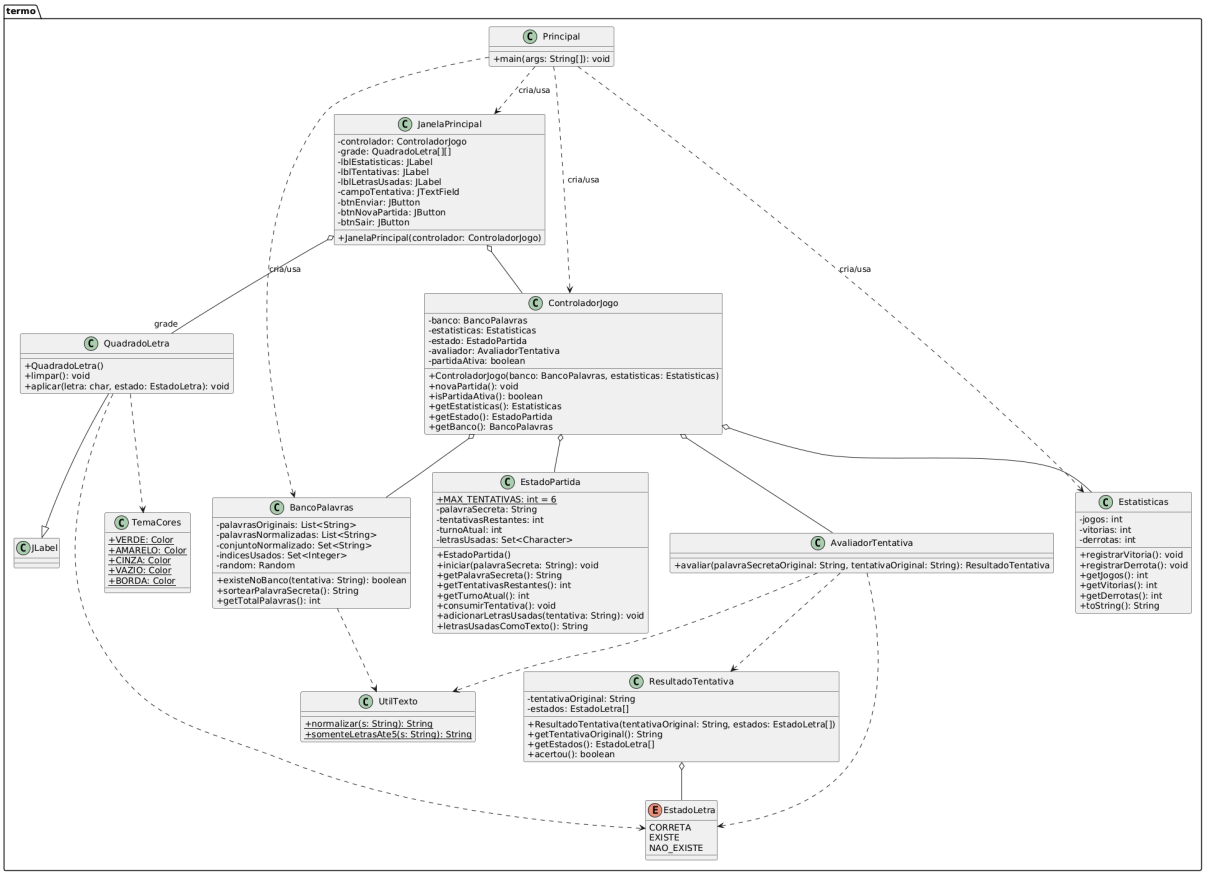

Figura 1 – Diagrama de Classes

DECISÕES DE PROJETO

- A modelagem do sistema foi estruturada com base nos princípios da Programação Orientada a Objetos:

  - **Encapsulamento:** Todos os atributos das classes são privados, permitindo acesso controlado por métodos públicos, garantindo integridade dos dados.

  - **Separação de Responsabilidades:** A lógica do jogo foi separada da interface gráfica. A classe Jogo contém apenas regras de negócio, enquanto JanelaPrincipal trata exclusivamente da interface.

● **Uso de Enumerações:** As classes EstadoLetra e EstadoPartida foram implementadas como enums para representar estados fixos e evitar uso de strings soltas.

● **Coleções Dinâmicas:** Foi utilizado ArrayList para armazenar palavras e tentativas, permitindo redimensionamento automático.

● **Manipulação de Arquivos:** A leitura do banco de palavras foi implementada utilizando BufferedReader e FileReader, garantindo eficiência na leitura linha a linha.

● **Interface com Swing:** Foram utilizados componentes como JFrame, JPanel, JButton, JLabel, JTextField e GridLayout para construção da interface gráfica. A escolha da biblioteca Swing se deve à sua integração nativa com Java e adequação ao escopo do trabalho..

TESTES E TRATAMENTO DE ERROS

Para validar o funcionamento do jogo TERMO, foram realizados testes manuais na interface gráfica, verificando tanto o fluxo principal da partida quanto situações de erro e entradas inválidas. Os testes confirmaram que o sistema responde corretamente com feedback visual no tabuleiro, atualização de estatísticas e exibição de mensagens ao usuário por meio de janelas de aviso (pop-ups).

Testes funcionais executados:

- **Execução de uma partida completa até derrota:** foi jogada uma sequência de tentativas até esgotar o limite, e o sistema encerrou a partida corretamente, exibindo uma mensagem de *Derrota* informando que as tentativas acabaram e revelando a palavra secreta (“Fim das tentativas! Palavra era: simio”).

- **Atualização de estatísticas:** após o término da partida, o sistema atualizou os contadores de partidas (Jogos), vitórias e derrotas no topo da tela,

refletindo corretamente o resultado acumulado.

- **Verificação de letras utilizadas:** a cada tentativa, o jogo adicionou as letras já informadas ao painel “Letras já utilizadas”, permitindo ao jogador acompanhar o histórico de letras usadas.

- **Feedback visual por cores:** em cada tentativa, as letras foram representadas no tabuleiro com as cores esperadas (verde para posição correta, amarelo para letra existente em posição errada e cinza para letra ausente), permitindo confirmar a consistência do avaliador de tentativas.

Tratamento de erros e entradas inválidas

O sistema implementa validações antes de processar uma tentativa, impedindo que entradas incorretas consumam tentativas e garantindo uma experiência consistente:

- **Palavra fora do banco:** ao digitar uma palavra que não existe no banco de palavras, o sistema exibe um aviso (“Palavra inválida (não está no banco).”), impedindo o processamento da tentativa.

- **Tamanho incorreto da palavra:** ao digitar uma palavra com menos de 5 letras (ex.: “casa” ou “CASA”), o sistema exibe um aviso (“Digite uma palavra com 5 letras.”), solicitando correção.

- **Fim da partida:** ao atingir o limite de tentativas, o sistema bloqueia a continuidade daquela rodada e exibe a tela de derrota, revelando a palavra secreta e garantindo o encerramento correto do estado da partida.

Esses testes demonstram que o sistema trata adequadamente os principais casos de erro (entrada inválida e palavra inexistente no banco) e mantém a

integridade da partida, evitando que o jogador avance com dados incorretos ou que perca tentativas injustamente.

CONCLUSÃO

O desenvolvimento do jogo TERMO permitiu a aplicação prática dos principais conceitos de Programação Orientada a Objetos em Java, integrando lógica de negócio, manipulação de arquivos e construção de interface gráfica.

A separação entre controle, lógica e interface contribuiu para um código mais organizado, modular e de fácil manutenção. A utilização de enums e coleções dinâmicas garantiu maior robustez ao sistema.

O sistema atende aos requisitos propostos, permitindo partidas completas com até seis tentativas, feedback visual por cores e controle de estatísticas.

Como melhorias futuras, sugere-se:

● Implementação do sistema de login e cadastro de usuários (tarefa extra);

● Persistência permanente das estatísticas em arquivo;

● Melhorias visuais na interface (animações e teclado virtual);

● Implementação de ranking entre jogadores